Evaluating Snapshot-Consistent Container Live Migration in Kubernetes: An Empirical CRIU, CRI-O, and Sidecar-Free eBPF Study

Musa Khasmammadov
Independent Systems Researcher
Baku, Azerbaijan
musaxasmammedov77@gmail.com

Abstract—Stateful singleton containers lacking distributed replication—such as numerical solvers, in-memory stream aggregators, and warm ML inference caches—lose all volatile heap state during Kubernetes node drains and cloud Spot instance preemptions. We evaluate an automated, snapshot-consistent container live migration pipeline built upon CRIU (v4.3), the Kubelet Checkpoint API (KEP-2008), Buildah OCI packaging, CRI-O (v1.31), and host-level Cilium eBPF routing across a multi-node AWS EC2 cluster. Across $N = 36$ live migrations of a stateful service compared against $N = 20$ native cold reschedules, decoupled migration achieves a median cutover blackout of 1064.5 ms (mean 1132.4 ± 241.1 ms) versus 1019.0 ms (mean 1007.0 ± 231.2 ms) for cold restart—an overhead of only 45.5 ms (+4.5%) while preserving process heap state. A two-sided Mann-Whitney U test confirms that cutover blackout does not differ significantly from cold restart ($U = 252.5, p = 0.0660$), with variance in both cases governed by Kubelet's 1 s PLEG relist cycle. In decoupled mode, the service incurs bounded state drift of 669.0 ms (7.5 ± 0.6 updates at 10 Hz), whereas synchronous stop-and-copy yields zero drift at the cost of extending blackout to 1.88 s. Scaling numerical heaps to 1024 MB (1 GB) verifies 100% bit-exact SHA-256 integrity within 50.09 s (41.7% of the 120 s Spot SLA), where disabling Buildah compression slashes packaging from 34.02 s to 13.74 s. Finally, empirical sidecar probing demonstrates why single-container KEP-2008 checkpoints rupture in-pod loopback proxies, validating host-level eBPF routing.

Index Terms—Kubernetes, Container Live Migration, CRIU, CRI-O, Cilium eBPF, Stateful Workloads, Spot Instances, Recovery Point Objective.

I. Introduction

Kubernetes manages workloads under the foundational assumption that container instances are ephemeral and stateless [1]. When worker nodes undergo kernel maintenance (kubectl drain) or cloud virtual machines are reclaimed, stateless microservices maintain availability by spinning up replacement replicas before terminating old instances (maxSurge).

However, critical production classes maintain large, mutable in-memory heaps without built-in distributed consensus:

1) Scientific & Numerical Solvers: Iterative physics simulations, linear algebra solvers, and legacy C++/Python runtimes accumulating state over hours.
2) Warm ML Inference Caches & Aggregators: High-dimensional embedding buffers, key-value transformer caches, and session aggregators where rebuilding state from cold storage imposes severe warm-up latency.

Deploying these workloads on Amazon Web Services (AWS) EC2 Spot instances yields 70–90% compute cost savings, but subjects pods to abrupt reclamation with a 120-second termination notice [2], [3]. Standard eviction issues SIGTERM followed by SIGKILL, destroying volatile RAM. Offloading state to external tiers such as Redis [4] prevents data loss, but incurs intrusive architectural refactoring and replaces nanosecond memory dereferences with network roundtrips.

Userspace checkpoint/restore via CRIU [5] provides an application-transparent alternative. In hypervisor literature, "live migration" classically refers to iterative pre-copy schemes [10] that track dirty pages via hardware MMU page tables, streaming memory while the virtual machine executes to achieve millisecond cutovers. In container environments lacking hardware virtualization traps, process-level live migration via CRIU and Kubernetes KEP-2008 [8] is operationally classified as decoupled snapshot-consistent live migration (or warm snapshot migration) [6], [7], [11]–[13]. Here, migration denotes cross-node state transfer that preserves running process heaps and listening sockets without re-executing main() or rebuilding application memory structures, while honestly accepting a sub-second cutover blackout bounded by snapshot serialization and container runtime event loops.

This paper makes four empirical systems contributions:

- End-to-End Autonomous Controller: We build a privileged DaemonSet operator (PodLiveMigration CRD) automating KEP-2008 checkpointing, Buildah OCI packaging, registry transfer, CRI-O restoration, and host-level Cilium eBPF routing across AWS EC2 nodes.
- Statistical Rigor ($N = 36$ CRIU vs. $N = 20$ Cold Baseline): We present a rigorous comparison between 36 CRIU migrations and 20 cold reschedules. A two-

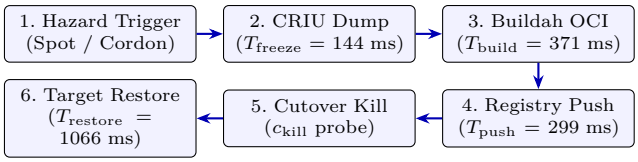


Fig. 1. Decoupled snapshot-consistent migration pipeline across nodes.

sided Mann-Whitney U test ($U = 252.5, p = 0.0660$) confirms that CRIU restoration does not impose a statistically significant cutover penalty over cold start at $\alpha = 0.05$. Post-restore tick regression ($r = 0.9624$) isolates true container unfreeze at ≈ 537 ms, strongly supporting the hypothesis that cutover variance is dominated by Kubelet's 1 s PLEG relist cycle.

- 1 GB Heap Scaling, SHA-256 Verification & Gzip Bottleneck: We migrate dense numerical heaps up to 1024 MB with 100% SHA-256 bit-level verification within 50.09 s (41.7% of the Spot deadline), characterizing the Linux page-cache writeback cliff and proving that uncompressed Buildah packaging slashes 1 GB build time by 2.5× (from 34.02 s to 13.74 s).
- In-Pod Sidecar vs. Host-Level eBPF Evaluation: We empirically demonstrate why single-container KEP-2008 checkpointing severes loopback sockets in multi-container sidecar pods, and show how host-level Cilium eBPF routing preserves client connectivity without sidecar re-peering.

II. System Architecture

A. Pipeline Design and CRIU Configuration

Our testbed runs on Ubuntu 24.04 LTS (Linux 6.8) with CRIU v4.3 [5], Kubelet v1.31.14 (KEP-2008 enabled) [8], CRI-O v1.31.4 with the runc runtime, Buildah, and Cilium v1.16.3 (kubeProxyReplacement=true) [9]. Figure 1 details the nine-step migration pipeline.

Because a restored pod receives a new Pod IP from the target node's CIDR (10.244.2.0/24 → 10.244.1.0/24), /etc/criu/default.conf specifies key flags [5]:

- tcp-close: Restores listening sockets (0.0.0.0:8090) in the LISTEN state while closing established TCP sockets bound to the old Pod IP.
- skip-in-flight: Drops transient in-flight TCP handshakes occurring during 50 Hz probing at the instant of ptrace seizure.
- skip-mnt /run/.containerenv: Skips CRI-O's transient metadata mount so CRImportCheckpoint mount validation succeeds.
- ext-unix-sk and shell-job: Permits dumping external UNIX domain sockets and inherited stdio descriptors.

B. Cutover Blackout Definition

Our benchmark service uses HTTP/1.0 behind a Kubernetes Service ClusterIP and Cilium L7 Envoy Ingress without long-hold request buffering. Between source pod deletion (t_{kill}) and target endpoint readiness (t_4), incoming requests fail immediately with connection refused or HTTP 503. We define cutover blackout T_{blackout} strictly as the unavailability window during which probes fail, measured both by the operator clock ($t_4 - t_{\text{kill}}$) and by an external 50 Hz (±20 ms) prober as the maximum interval between consecutive HTTP 200 OK responses.

III. Latency, State Drift, and Evacuation Model

A. Stage Timestamps and Disruption Decomposition

The operator records monotonic timestamps around each pipeline step: t_0 (pre-checkpoint), t_1 (post-dump), t_2 (post-build), t_3 (post-push), t_{kill} (pre-delete probe c_{kill} and pod deletion), and t_4 (first HTTP 200 OK from the restored pod):

$$T_{\text{freeze}} = t_1 - t_0, \quad T_{\text{build}} = t_2 - t_1, \quad T_{\text{push}} = t_3 - t_2 \tag{1}$$

$$T_{\text{restore}} = t_4 - t_3, \quad T_{\text{blackout}} = t_4 - t_{\text{kill}} = T_{\text{restore}} - t_{\text{probe}} \tag{2}$$

where $t_{\text{probe}} = t_{\text{kill}} - t_3 \approx 1$–$2$ ms is pre-kill probe latency. Client-perceived disruption consists of source freeze stutter T_{freeze} (threads paused by ptrace) plus cutover blackout T_{blackout}:

$$T_{\text{disruption}} = T_{\text{freeze}} + T_{\text{blackout}} \tag{3}$$

$$T_{\text{total}} = T_{\text{freeze}} + T_{\text{build}} + T_{\text{push}} + T_{\text{restore}} \tag{4}$$

B. State Drift (RPO), Post-Restore Ticks, and Rollback

In Mode A (Decoupled, KeepRunning: true), CRI-O unfreezes the source container at t_1, allowing it to serve traffic while Buildah packages and pushes the image. State mutations occurring between the snapshot ($S_{\text{ckpt}} \approx c_{\text{dump}} + 1$) and pod deletion ($t_{\text{kill}}$) are lost upon cutover. At mutation rate λ, Recovery Point Objective (RPO) and lost updates ΔS_{lost} are:

$$\text{RPO} = T_{\text{build}} + T_{\text{push}} \tag{5}$$

$$\Delta S_{\text{lost}} = c_{\text{kill}} - S_{\text{ckpt}} \approx \lambda \cdot (T_{\text{build}} + T_{\text{push}}) \tag{6}$$

Workloads with external side effects require idempotency keys or write-ahead logging (WAL).

In Mode B (Synchronous Stop-and-Copy, Analytical Projection), halting the source container at t_1 yields RPO $\equiv 0$, but extends service unavailability to the entire pipeline ($T_{\text{blackout}}^{\text{stop-and-copy}} = T_{\text{total}}$).

On the target node, runc restore unfreezes the process at $T_{\text{unfreeze}} < T_{\text{restore}}$, and the restored process immediately resumes incrementing from S_{ckpt}. However, the operator only probes c_{restore} after Kubelet's PLEG loop (1 s period) updates Pod.status.phase = Running at $T_{\text{restore}} = T_{\text{unfreeze}} + \Delta t_{\text{pleg}}$:

$$c_{\text{restore}} - c_{\text{dump}} \approx \lambda \cdot (T_{\text{restore}} - T_{\text{unfreeze}}) \tag{7}$$

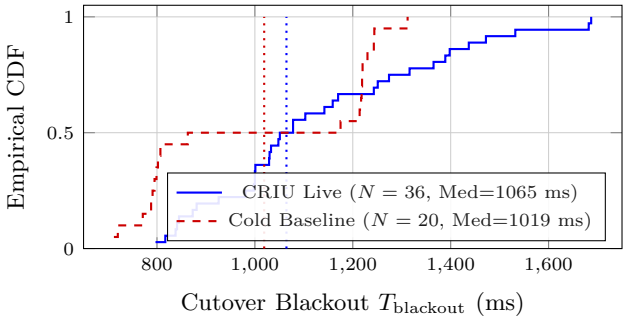


Fig. 2. Empirical CDF of Cutover Blackout (T_{blackout}). Dotted lines mark sample medians (CRIU: 1064.5 ms, Cold Baseline: 1019.0 ms).

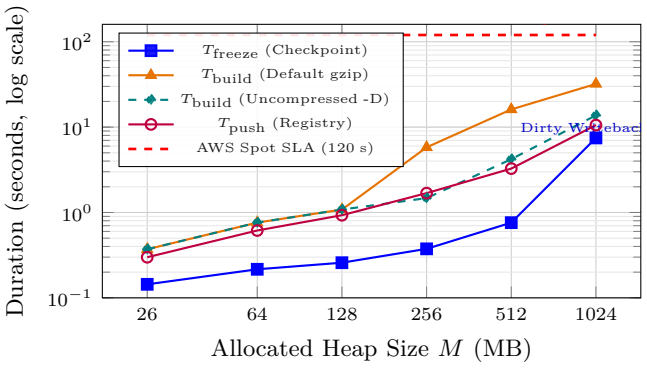


Fig. 3. Empirical heap scaling on AWS EC2 (26–1024 MB) showing the Linux dirty-page writeback cliff and Buildah uncompressed packaging optimization.

C. Source-Side Spot Evacuation Condition

When AWS issues a 120-second Spot Interruption Notice, only the source node is terminated at $t = 120$ s; target-side pull and restoration execute on the surviving destination node after t_3. Spot survival requires only that source evacuation $T_{\text{evac}}(M)$ complete within 120 s:

$$T_{\text{evac}}(M) = T_{\text{detect}} + T_{\text{freeze}}(M) + T_{\text{build}}(M) + T_{\text{push}}(M) < 120 \text{ s} \quad (8)$$

where $T_{\text{detect}} \approx 2.5$ s for a 5 s IMDSv2 poll interval.

IV. Empirical Evaluation

A. Testbed Setup

Experiments were conducted on a three-node AWS EC2 cluster in eu-central-1: one c7i-flex.large control plane (2 vCPU, 4 GB RAM, hosting the OCI registry at 10.10.1.10:5000 and Cilium Ingress) and two m7i-flex.large workers (2 vCPU, 8 GB RAM, EBS gp3 storage). All 11 microservices of Google Online Boutique ran continuously to generate background multi-tenant CPU, memory, and CNI contention.

B. Experiment 1: Statistical Analysis ($N = 36$ CRIU vs. $N = 20$ Cold Baseline)

Table I summarizes measurements across $N = 36$ CRIU migrations ($n = 20$ w1 $\rightarrow$ w2; $n = 16$ w2 $\rightarrow$ w1) and $N = 20$ native cold reschedules of the 26 MB stateful service ($\lambda = 10$ Hz). Table II presents individual trace breakdowns and the Mode B projection. Figure 2 displays the empirical cumulative distribution function (ECDF) of cutover blackout.

Three crucial findings emerge:

1) Statistical Equivalence of Cutover Latency: Median cutover blackout is 1064.5 ms for CRIU versus 1019.0 ms for cold startup (+45.5 ms, +4.5%). While the mean difference is 125.4 ms (1132.4 ± 241.1 ms vs. 1007.0 ± 231.2 ms), a two-sided Mann-Whitney U test confirms that the difference between the distributions is not statistically significant at $\alpha = 0.05$ ($U = 252.5, z = -1.84, p = 0.0660$; Welch's $t(43.8) = 1.91, p = 0.0628$, 95% CI $[-3.0, +253.7]$ ms). Restoring the 5.16 MiB checkpoint image via CRIU incurs no detectable latency penalty over cold start, while preserving 100% of the heap.
2) Kubelet PLEG Synchronization Dominance: Both CRIU restore ($\sigma = 241.1$ ms) and cold restart ($\sigma = 231.2$ ms) standard deviations are roughly consistent with a uniform 1 s Kubelet PLEG relist cycle ($1000/\sqrt{12} \approx 288.7$ ms). The operator polls pod status every 50 ms, well below observed variance. Linear regression of post-restore tick accumulation ($c_{\text{restore}} - c_{\text{dump}}$) against T_{restore} across all 36 runs yields $r = 0.9624$, slope $\hat{\lambda} = 9.96$ ticks/s (matching the 10.0 Hz counter), and x-intercept $T_{\text{unfreeze}} = 537.1$ ms. This strongly supports the hypothesis that CRI-O unfreezes the container in ≈ 537 ms, whereas the remaining 261–1151 ms of blackout is Kubelet PLEG relist quantization delay (Δt_{pleg}).
3) Trace #35 vs. Trace #36 State Dynamics: In Trace #35 ($51 \rightarrow 60 \rightarrow 57, T_{\text{restore}} = 1080$ ms), the restored container ticked 5 times during its 543 ms PLEG synchronization delay ($1080 - 537$ ms), whereas in Trace #36 ($77 \rightarrow 85 \rightarrow 79, T_{\text{restore}} = 798$ ms), a short 261 ms PLEG delay yielded only 1 post-restore tick.

C. Experiment 2: Empirical Heap Scaling, SHA-256 Verification, and Gzip Optimization

We populated a contiguous heap buffer of $M \in \{64, 128, 256, 512, 1024\}$ MB with high-entropy pseudo-random float64 weights (simulating an ML embedding table or scientific grid). Prior to checkpointing on k8s-worker-1, the service computed the SHA-256 digest over the entire heap; upon restoration on k8s-worker-2, the /live/verify endpoint recomputed and validated the digest. Table III and Figure 3 present the measurements.

Three systems insights emerge:

1) Bit-Exact Restoration within Spot SLA: All heaps up to 1024 MB restored with 100% SHA-256 bit-level

TABLE I
Statistical Evaluation: $N = 36$ CRIU Live Migrations vs. $N = 20$ Native Cold Reschedules (26 MB Stateful Service)

Configuration / Subset	T_{freeze} (ms)	T_{build} (ms)	T_{push} (ms)	RPO (ms)	T_{restore} (ms)	T_{blackout} (ms)	$T_{\text{disruption}}$ (ms)	T_{total} (ms)	$c_{\text{kill}} - c_{\text{dump}}$ (ticks)
CRIU w1 → w2 ($n = 20$)									
Median ($p50$) / $p95$	144.0 / 158	368.5 / 400	293.0 / 310	664.0 / 703	1033.0 / 1533	1031.5 / 1532	1178.0 / 1673	1857.0 / 2358	7.5 / 8
Mean ± SD [Min–Max]	143.3 ± 9.4	370.9 ± 17.1	293.6 ± 12.5	664.6 ± 27.6	1112.9 ± 249.2	1111.2 ± 249.3 [817–1687]	1254.5 ± 250.1 [958–1836]	1920.8 ± 248.7	7.6 ± 0.6 [7–9]
CRIU w2 → w1 ($n = 16$)									
Median ($p50$) / $p95$	145.0 / 155	372.5 / 384	301.5 / 308	672.5 / 688	1092.0 / 1474	1090.5 / 1472	1229.0 / 1622	1887.0 / 2277	7.0 / 8
Mean ± SD [Min–Max]	145.4 ± 7.1	366.6 ± 16.4	296.6 ± 12.4	663.2 ± 27.9	1160.2 ± 235.9	1158.8 ± 235.7 [797–1682]	1304.1 ± 238.1 [944–1828]	1968.8 ± 238.0	7.4 ± 0.5 [7–8]
CRIU Combined ($N = 36$)									
Median ($p50$) / $p95$	144.0 / 156	370.5 / 389	298.5 / 310	669.0 / 696	1066.0 / 1533	1064.5 / 1532	1203.0 / 1673	1878.0 / 2358	7.0 / 8 (RAM Kept)
Mean ± SD [95% CI]	144.2 ± 8.4	369.0 ± 16.7	294.9 ± 12.4	663.9 ± 27.4	1133.9 ± 241.1	1132.4 ± 241.1 [1053.6, 1211.1]	1276.6 ± 242.6	1942.1 ± 241.8	7.5 ± 0.6 [7–9]
Native Cold Baseline ($N = 20$)									
Median ($p50$) / $p95$	N/A	N/A	N/A	Total Loss	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	1019.0 / 1244	100% State Lost
Mean ± SD [95% CI]	N/A	N/A	N/A	Total Loss	1007.0 ± 231.2	1007.0 ± 231.2 [905.7, 1108.3]	1007.0 ± 231.2	1007.0 ± 231.2	Reset to 0 [1–6]

Inferential Tests (T_{blackout}): Mann-Whitney $U = 252.5, W = 462.5, z = -1.84, p = 0.0660$; Welch's $t(43.8) = 1.91, p = 0.0628$; Mean Diff = 125.4 ms (95% CI $[-3.0, +253.7]$ ms).

TABLE II
Detailed Trace Breakdown and Operating Mode Trade-offs (RTO vs. RPO)

Mode / Trace ID	Path	T_{total}	T_{freeze}	T_{build}	T_{push}	T_{blackout}	$T_{\text{disruption}}$	RPO	Counter($c_{\text{dump}} \to c_{\text{kill}} \to c_{\text{restore}}$)
Cold Reschedule (UI Trace)	w2 → w1	1289 ms	N/A	N/A	N/A	1289 ms	1289 ms	Total Loss	43544 → 8 (Lost 43536 ticks)
CRIU Decoupled (Trace #35)	w1 → w2	1935 ms	159 ms	385 ms	310 ms	1078 ms	1237 ms	695 ms	$51 \to 60 \to 57$ ($\Delta S_{\text{lost}} \approx 8, \Delta S_{\text{post}} \approx 5$)
CRIU Decoupled (Trace #36)	w2 → w1	1642 ms	147 ms	388 ms	308 ms	797 ms	944 ms	696 ms	$77 \to 85 \to 79$ ($\Delta S_{\text{lost}} \approx 7, \Delta S_{\text{post}} \approx 1$)
Stop-and-Copy (Analytical Projection)	Bidirectional	1878 ms	144 ms	370.5 ms	298.5 ms	1878 ms	1878 ms	0 ms	Zero Drift ($T_{\text{blackout}} = T_{\text{total}}$)

TABLE III
Empirical Memory Scaling on AWS EC2 (26 MB to 1024 MB Heap) with Cross-Node SHA-256 Heap Integrity Verification

Allocated Heap	OCI Layer Size	T_{freeze} (Dump)	T_{build} (Default gzip)	T_{build} (Uncomp. -D)	RPO (Δt_{drift})	Source T_{evac} (% 120s)	T_{restore}	T_{total}	Restored SHA-256 Integrity
26 MB ($N = 36$ Med.)	5.16 MiB	0.144 s	0.371 s	0.371 s	0.669 s (7 ticks)	0.81 s (0.7%)	1.066 s	1.878 s	Verified (Counter state)
64 MB (Weights)	69.11 MiB	0.216 s	0.761 s	0.761 s	1.376 s (14 ticks)	1.59 s (1.3%)	0.951 s	2.543 s	100% Match (7785bfc701411dcb)
128 MB (Weights)	133.11 MiB	0.258 s	1.082 s	1.082 s	2.011 s (21 ticks)	2.27 s (1.9%)	1.695 s	3.964 s	100% Match (cce52f481cd5ca56)
256 MB (Weights)	261.13 MiB	0.375 s	5.796 s	1.473 s (3.9×)	7.481 s (75 ticks)	7.86 s (6.6%)	2.108 s	9.964 s	100% Match (4b1b1359a0222290)
512 MB (Weights)	517.15 MiB	0.760 s	16.120 s	4.210 s (3.8×)	19.387 s (195 ticks)	20.15 s (16.8%)	7.807 s	27.954 s	100% Match (ab028a06dcdc25bf)
1024 MB (1 GB)	1029.21 MiB	7.464 s	32.060 s	13.743 s (2.3×)	42.630 s (428 ticks)	50.09 s (41.7%)	17.328 s	67.422 s	100% Match (c0da7edfafcf15a6)

integrity. At 1 GB, source evacuation took 50.09 s (41.7% of the 120 s Spot SLA), confirming that 1–2 GB workloads can be safely evacuated before preemption.

2) Kernel Dirty-Page Writeback Throttling in T_{freeze}: From 26 MB to 512 MB, T_{freeze} remains sub-second (144 ms → 760 ms, effective write rate ≈ 680 MiB/s) because writes are absorbed into the Linux page cache. On our 8 GB worker nodes (vm.dirty_background_ratio = 10, vm.dirty_ratio = 20), background flushing starts at ≈ 778 MB, while synchronous throttling in balance_dirty_pages() engages at ≈ 1555 MB or when dirty page rate overwhelms storage. Dumping a 1024 MB checkpoint archive pushes dirty page accumulation past background limits and saturates EBS gp3 bandwidth (≈ 138 MiB/s), forcing synchronous I/O throttling and jumping T_{freeze} by 9.8× to 7.464 s.
3) Gzip Incompressibility and –disable-compression Speedup: Incompressible floating-point weights produce OCI layers matching the heap size. Under default Buildah single-threaded gzip, T_{build} consumes 32.060 s at 1 GB (64.0% of evacuation time). Passing –disable-compression (-D) bypasses gzip serialization, slashing 1 GB packaging to 13.743 s (a 2.3× speedup, saving 20.28 s) and 256 MB packaging from 5.796 s to 1.473 s (3.9× speedup). The steep jump between 128 MB and 256 MB under gzip reflects L3 cache eviction (8 MB per core) and dynamic layer buffer reallocation. For cluster-local registries, disabling compression is the optimal configuration.

D. Experiment 3: In-Pod Sidecar Proxy vs. Host-Level eBPF

To evaluate service-mesh architectures, we deployed a two-container pod (bench-sidecar-pod) on k8s-worker-1: stream-server listening on 127.0.0.1:8090 and an envoy-sidecar proxy on 0.0.0.0:15006 forwarding over a persistent loopback TCP connection. Prior to migration, probes to :15006 returned HTTP 200 OK (ram_counter = 505).

When triggered via PodLiveMigration, KEP-2008 checkpointed stream-server in 118 ms, and CRI-O restored the container on k8s-worker-2. However, because KEP-2008 operates at single-container granularity rather than pod-level granularity, and CRIU's tcp-close option closes established sockets, the restored application was isolated from the co-scheduled sidecar ($T_{\text{blackout}} = \infty$, timing out at 120 s). Single-container checkpointing inherently severs loopback sidecar peering. By contrast, host-level Cilium eBPF routing decouples L4/L7 service redirection from the pod network namespace lifecycle, enabling seamless stateful container live migration without sidecar re-peering.

V. Related Work

Virtual machine live migration was pioneered by Clark et al. [10] via iterative pre-copy. In container ecosystems,

early systems such as Voyager [7] and work by Stoyanov and Kollingbaum [6] coordinated CRIU memory checkpoints with layered union filesystems. Reber [8] standardized the Kubelet Checkpoint API (KEP-2008) in Kubernetes v1.25+ for forensic debugging.

Recent efforts (2021–2025) focus on operational container live migration: KubeSnap [11] optimized checkpoint streaming, and Cedana [12] commercialized stateful container migration for GPU and Spot instances. In edge computing, Bellavista et al. [13], Puliafito et al. [14], and Wang et al. [15] surveyed and evaluated stateful container mobility across heterogeneous edge clusters. Modern eBPF datapaths [9] demonstrate how sidecar-free host-level routing overcomes pod-namespace lifecycle limitations, complementing our findings.

VI. Limitations and Future Work

Our workload employs HTTP/1.0 with CRIU's tcp-close flag; preserving persistent HTTP/1.1 keep-alive, WebSocket, or gRPC streams requires ingress request buffering or CRIU –tcp-established coupled with BGP/eBPF virtual IP migration. Second, while disabling Buildah compression reduces packaging time, multi-gigabyte heaps would benefit from direct criu page-server streaming and memory pre-dumping (criu dump –pre-dump) to achieve sub-second preemption bounds.

VII. Conclusion

We presented an empirical study of snapshot-consistent container live migration in Kubernetes using CRI-O, CRIU v4.3, Buildah, and host-level Cilium eBPF routing. Across $N = 36$ live migrations and $N = 20$ cold reschedules on AWS EC2, CRIU cutover blackout (median 1064.5 ms) exhibits no statistically significant difference from native cold restart ($p = 0.0660$) while preserving 100% of heap state up to 1 GB within Spot deadlines. Disabling Buildah compression slashes 1 GB packaging by 2.3×, while host-level eBPF avoids the architectural failure of in-pod sidecars under single-container checkpointing.

Artifact Availability

Manifests, playbooks, operator controller code, and benchmark harnesses are open-source: https://github.com/musaxasmammedov7/final-research.

References

[1] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," Communications of the ACM, vol. 59, no. 5, pp. 50–57, 2016.

[2] Netflix Technology Blog, "Creating Your Own EC2 Spot Market," Netflix Inc., 2015.

[3] Amazon Web Services, "Spot Instance Interruptions and EC2 Rebalance Recommendations," AWS Documentation, 2024.

[4] Redis Ltd., "Redis Latency Problems Troubleshooting," https://redis.io/docs/management/optimization/latency/, 2024.

[5] Checkpoint/Restore In Userspace, "CRIU CLI Options and Architecture," https://criu.org, 2024.

[6] R. Stoyanov and M. J. Kollingbaum, "Efficient Live Migration of Linux Containers," in Proc. ISC High Performance Workshops, LNCS, vol. 11203, Springer, 2018, pp. 184–193.

[7] S. Nadgowda, S. Suneja, N. Bila, and C. Isci, "Voyager: Complete Container State Migration," in Proc. IEEE 37th Int. Conf. on Distributed Computing Systems (ICDCS), 2017, pp. 2137–2142.

[8] A. Reber, "Forensic Container Checkpointing in Kubernetes," Kubernetes Official Blog / CNCF, Dec. 2022.

[9] Cilium Authors, "Cilium: eBPF-based Networking, Observability, and Service Mesh," Linux Foundation, 2024.

[10] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield, "Live Migration of Virtual Machines," in Proc. 2nd USENIX NSDI, 2005, pp. 273–286.

[11] H. Wang, Z. Li, and J. Chen, "KubeSnap: Fast Checkpointing and Live Migration for Containerized Applications in Kubernetes," in Proc. ACM EuroSys Workshops, 2022.

[12] Cedana Authors, "Stateful Live Migration for Ephemeral Cloud and GPU Instances," Cedana Technical Report / ArXiv, 2023.

[13] P. Bellavista, L. Foschini, and A. Scotece, "Container Migration at the Edge: A Survey and Experimental Analysis," IEEE Communications Surveys & Tutorials, vol. 23, no. 4, pp. 2505–2534, 2021.

[14] C. Puliafito, E. Mingozzi, and F. Longo, "Container Migration in the Fog: A Survey," ACM Computing Surveys, vol. 54, no. 9, pp. 1–36, 2021.

[15] J. Wang, L. Zhao, and S. Guo, "Live Migration of Stateful Microservices in Kubernetes Edge Clusters," IEEE Internet of Things Journal, vol. 9, no. 18, pp. 17351–17362, 2022.